

Supervised Classification: Neural Networks

The Magic Wand of Artificial Intelligence

Maxime Boucher – Yannick Voglaire

ICHEC - Brussels Management School,
Laboratoire QUARESMI
Site d'Anjou, Rue au Bois, 365a
QUARESMI : bureau B307
1150 Bruxelles

Neural Networks



- 1 Introduction
- 2 Neural Networks
- 3 Tuning Parameters
- 4 Toy Example



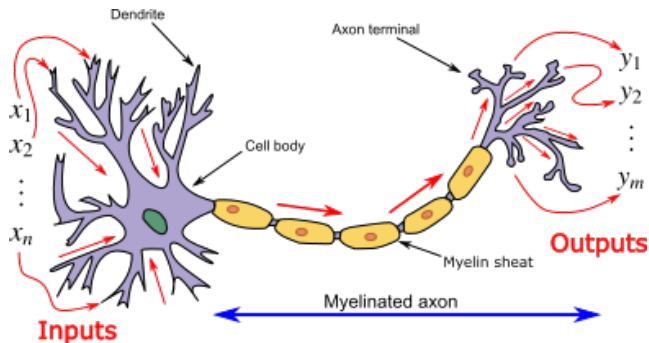
- 1 Introduction
- 2 Neural Networks
- 3 Tuning Parameters
- 4 Toy Example



1 Introduction

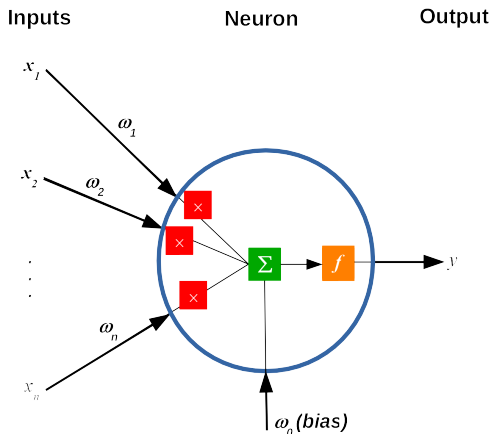
- Artificial Neuron
- Activation Functions



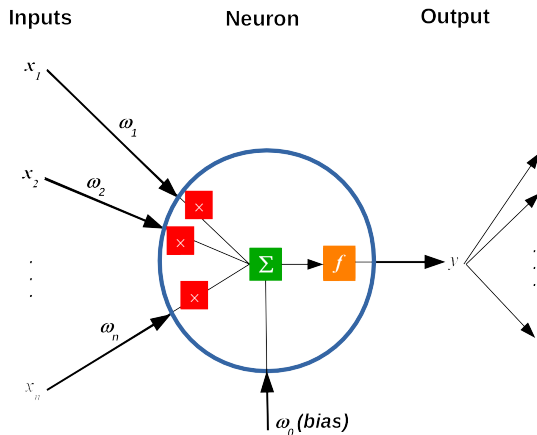


Real neurons are connected and exchange signals.

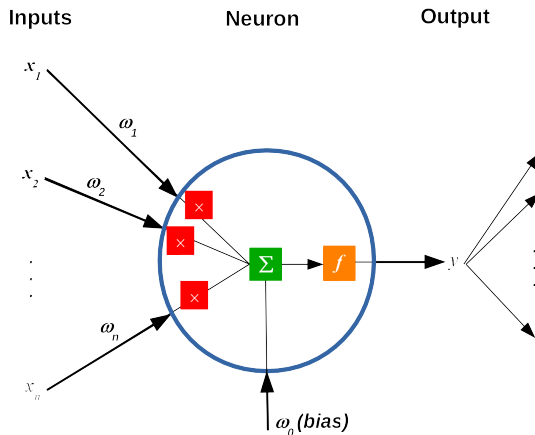
Source : <https://commons.wikimedia.org/wiki/File:Neuron3.png>



Artificial neurons receive inputs and do some calculations to produce an output.



Artificial neurons receive inputs and do some calculations to produce an output.



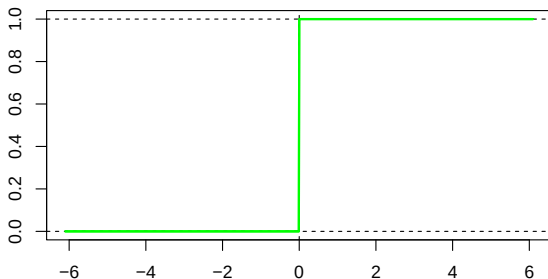
$$y = f \left(\omega_0 + \sum_{i=1}^n \omega_i \cdot x_i \right)$$

1 Introduction

- Artificial Neuron
- Activation Functions

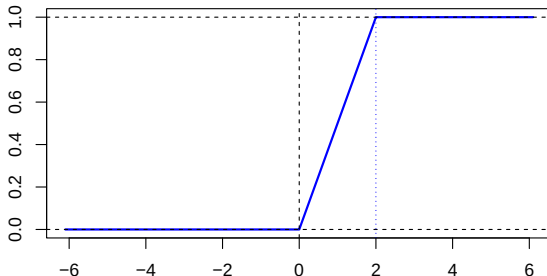


Activation functions : Threshold



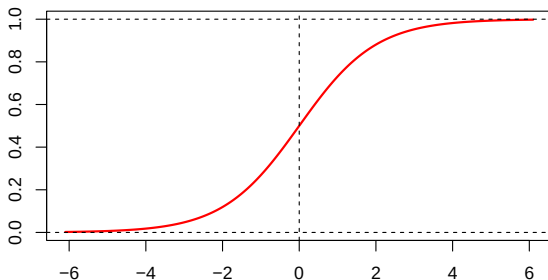
$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \end{cases}$$

Activation functions : Saturation



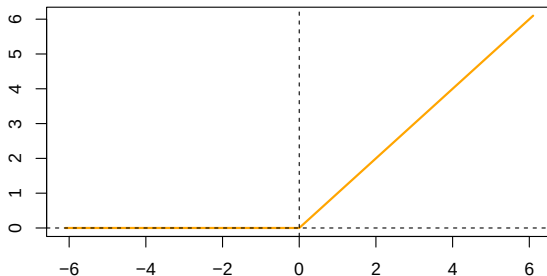
$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ x/a & \text{if } x \in [0, a] \\ 1 & \text{if } x > a \end{cases}$$

Activation functions : Sigmoid



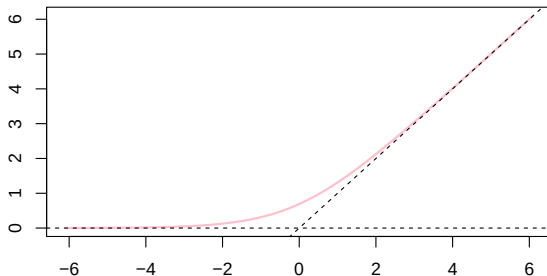
$$f_s(x) = \frac{1}{1 + e^{-a \cdot x}}$$

Activation functions : ReLU



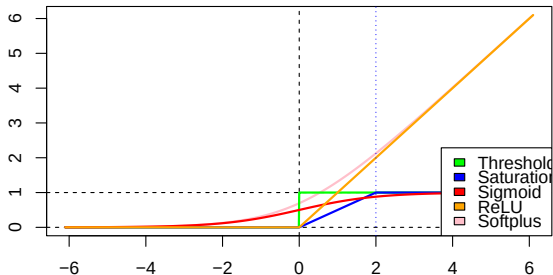
$$f(x) = \max(0, x)$$

Activation functions : Softplus



$$f(x) = \ln(1 + \exp(x))$$

Activation functions



- 1 Introduction
- 2 Neural Networks
- 3 Tuning Parameters
- 4 Toy Example



- 2 Neural Networks
 - General Examples
 - Feedforward



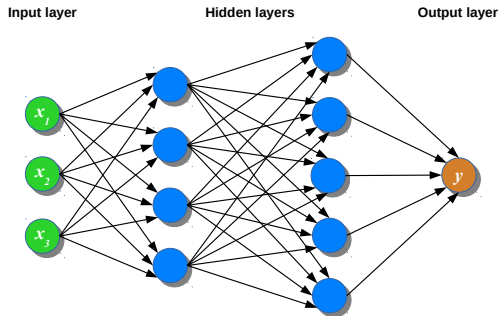


Figure: One type of neural networks : multilayer perceptron

https://en.wikipedia.org/wiki/Types_of_artificial_neural_networks



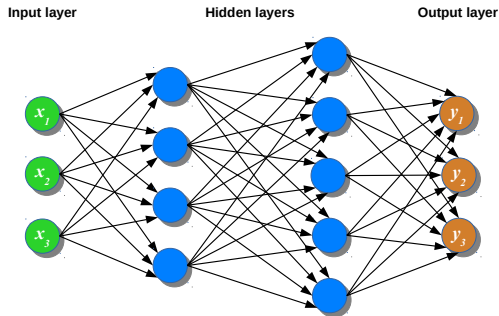


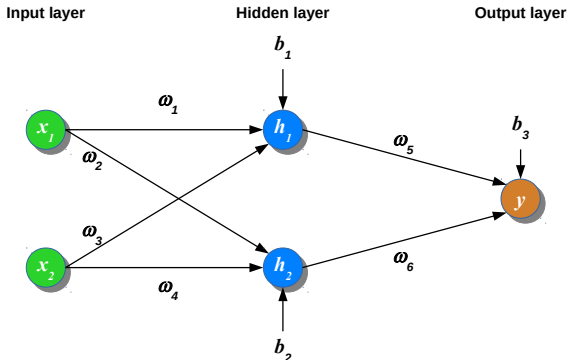
Figure: One type of neural networks : multilayer perceptron

https://en.wikipedia.org/wiki/Types_of_artificial_neural_networks



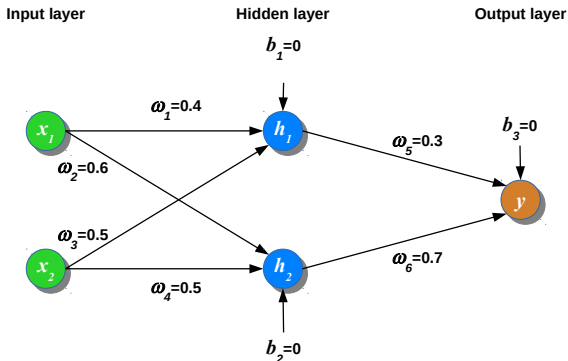
- 2 Neural Networks
 - General Examples
 - Feedforward



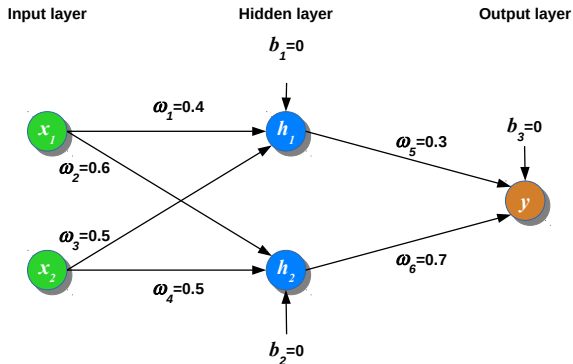


The following example is freely inspired by this post: [link](#).

Let the use the following (and random) parameters :

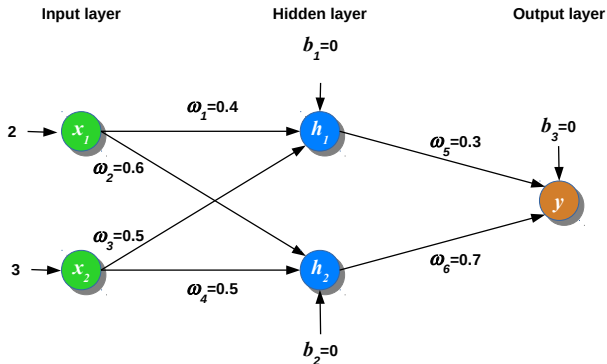


Let the following (and random) parameters :



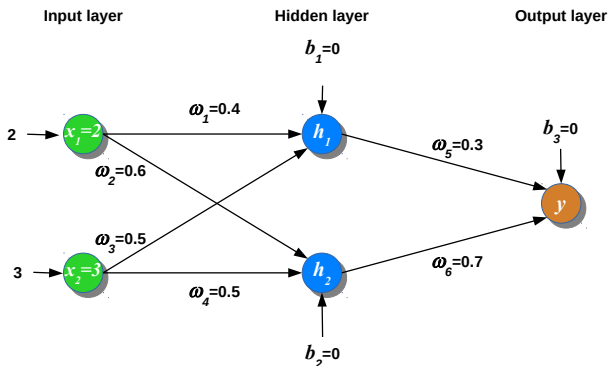
and do some computations...

Let the following (and random) parameters :



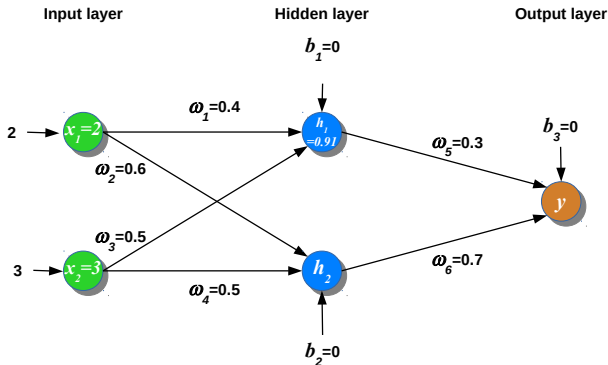
and do some computations...

Let the following (and random) parameters and the sigmoid activation function f_s :



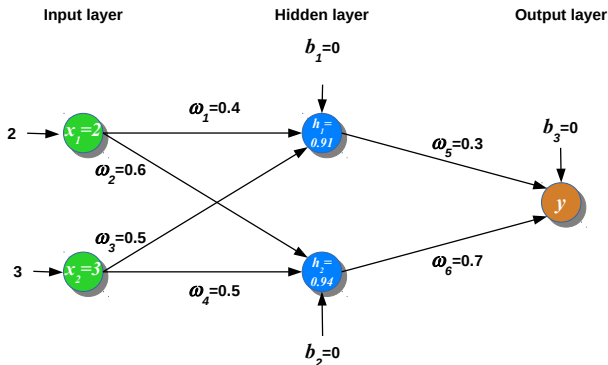
$$h_1 = f_s(x_1 \cdot \omega_1 + x_2 \cdot \omega_3 + b_1) = f_s(2 \cdot 0.4 + 3 \cdot 0.5 + 0) = f_s(2.3) = 0.909$$

Let the following (and random) parameters and the sigmoid activation function f_s :



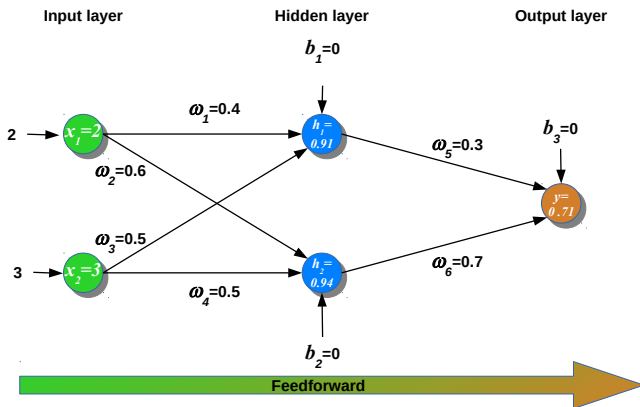
$$h_2 = f_s(x_1 \cdot \omega_2 + x_2 \cdot \omega_4 + b_2) = f_s(2 \cdot 0.6 + 3 \cdot 0.5 + 0) = f_s(2.7) = 0.937$$

Let the following (and random) parameters and the sigmoid activation function f_s :



$$y = f_s(h_1 \cdot \omega_5 + h_2 \cdot \omega_6 + b_3) = f_s(0.909 \cdot 0.3 + 0.937 \cdot 0.7 + 0) = f_s(0.92) = 0.716$$

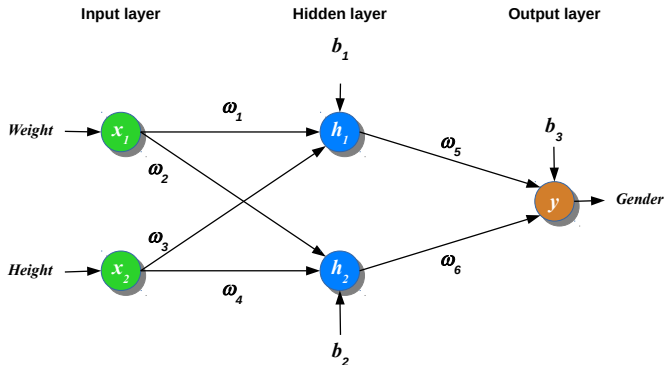
Let the following (and random) parameters and the sigmoid activation function f_s :



The process of passing inputs forward to get an output is called **feedforward**.

- 1 Introduction
- 2 Neural Networks
- 3 Tuning Parameters
- 4 Toy Example

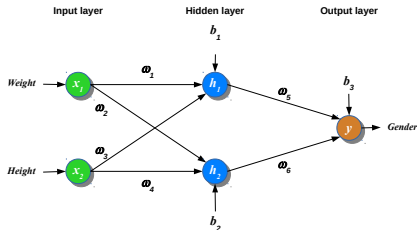




Given the data

Name	Weight(lb)	Height(in)	Gender
Alice	133	65	F
Bob	160	72	M
Charlie	152	70	M
Diana	120	60	F

Find the best parameters ($\omega_1, \omega_2, \omega_3, \omega_4, \omega_5, \omega_6, b_1, b_2, b_3$)

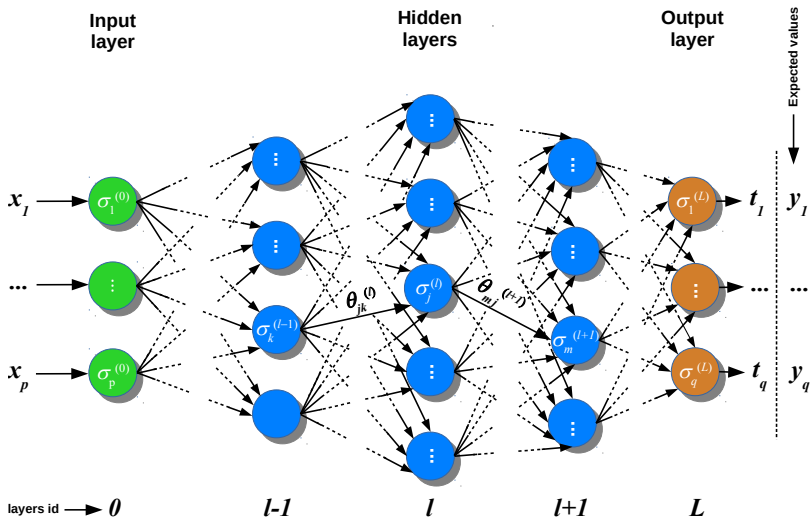


such given a weight and an height we can decide the gender
...most of the time.

3 Tuning Parameters

- General Framework
- Loss Function
- Stochastic Gradient Descent
- Backward Propagation





Let

- a dataset with
 - N items (data, rows,...),
 - p independent variables $x_1, \dots, x_p$ ($x_{1,i}, \dots, x_{p,i}$ for the i^{th} item),
 - a dependent variable Y with q possibles values $y_1, \dots, y_q$ ($y_{1,i}, \dots, y_{q,i}$ for the i^{th} item),
- a perceptron with
 - L layers of neurons,
 - p input cells (seen as layer 0),
 - the j^{th} neuron of the layer l denoted $\sigma_j^{(l)}$ (as well its output),
 - the weight between $\sigma_j^{(l)}$ and $\sigma_k^{(l-1)}$ denoted $\theta_{jk}^{(l)}$,
 - $b_j^{(l)}$ the bias used for $\sigma_j^{(l)}$,
 - q outputs cells¹ on layer L ,
 - $t_j = \sigma_j^{(L)}$, the j^{th} output value,
 - $\Theta = \left(\theta_{11}^{(1)}, \dots, \theta_{n_l n_{l-1}}^{(1)}, b_1^{(1)}, \dots, \theta_{q n_{L-1}}^{(L)}, b_q^{(L)} \right)^T$.

¹In the case of $q = 2$, we can consider that the response is binary with values in $\{0, 1\}$, or if Y is a real value with values in a interval $[a, b]$, then a single output cell could be used.

3 Tuning Parameters

- General Framework
- **Loss Function**
- Stochastic Gradient Descent
- Backward Propagation



To evaluate the “correctness” of output results $(t_{1,i}, \dots, t_{q,i})$ compared to the real value(s) $(y_{1,i}, \dots, y_{q,i})$ when the network is fed with the values $x_{1,i}, \dots, x_{p,i}$ of the i^{th} item, we define E_i , the **loss function** for the i^{th} item:

$$E_i(\Theta) = \sum_{r=1}^q (t_{r,i} - y_{r,i})^2$$

and, for the complete dataset, the **average loss function**

$$E(\Theta) = \frac{1}{N} \sum_{i=1}^N E_i(\Theta)$$

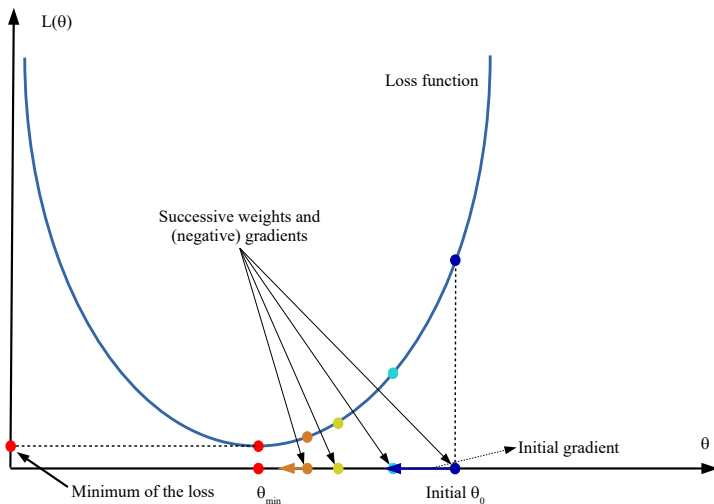
and we search for Θ which minimize the loss.



3 Tuning Parameters

- General Framework
- Loss Function
- Stochastic Gradient Descent
- Backward Propagation





General principle: repeat

$$\Theta \leftarrow \Theta - \eta \nabla E(\Theta) = \Theta - \frac{\eta}{N} \sum_{i=1}^N \nabla E_i(\Theta)$$

i.e.

$$\Theta \leftarrow \begin{pmatrix} \theta_{11}^{(1)} \\ b_1^{(1)} \\ \vdots \\ \theta_{qn_{L-1}}^{(L)} \\ \partial b_q^{(L)} \end{pmatrix} - \frac{\eta}{N} \sum_{i=1}^N \begin{pmatrix} \frac{\partial E_i}{\partial \theta_{11}^{(1)}} \\ \frac{\partial E_i}{\partial b_1^{(1)}} \\ \vdots \\ \frac{\partial E_i}{\partial \theta_{qn_{L-1}}^{(L)}} \\ \frac{\partial E_i}{\partial b_q^{(L)}} \end{pmatrix}$$

and η is a constant called the **learning rate** that controls how fast we train.



Gradient descent algorithm :

Set $MaxStep, \eta$

$i \leftarrow 0$

repeat

Compute: $\nabla E(\Theta) = \frac{1}{N} \sum_{i=1}^N \nabla E_i(\Theta)$

Update Θ :

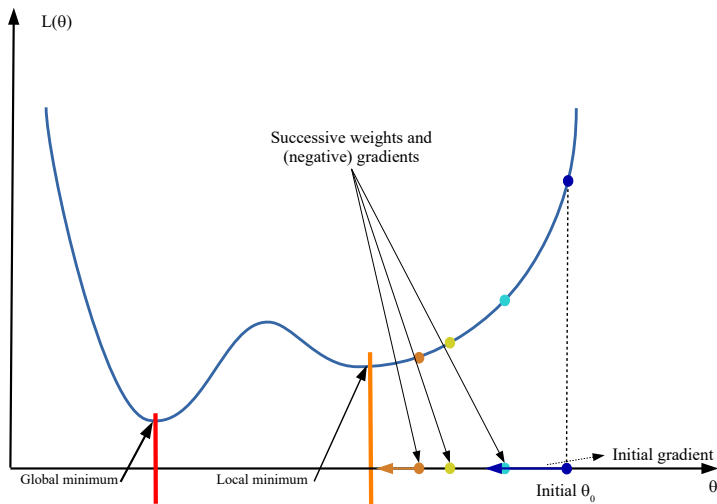
$$\Theta \leftarrow \Theta - \eta \nabla E(\Theta)$$

$i \leftarrow i + 1$

until (convergence) OR ($i \geq MaxStep$)

It could be heavy in calculations because you have to run through **all** the dataset to do a single update for a parameter in a particular iteration.





Stochastic gradient descent algorithm :

Set $MaxStep, \eta$

$i \leftarrow 0$

Set $b \in \{1, \dots, N\}$

repeat

Draw a sample $\mathcal{B}$ of size b from dataset

 Compute: $\nabla E_{\mathcal{B}}(\Theta) = \frac{1}{b} \sum_{i \in \mathcal{B}} \nabla E_i(\Theta)$

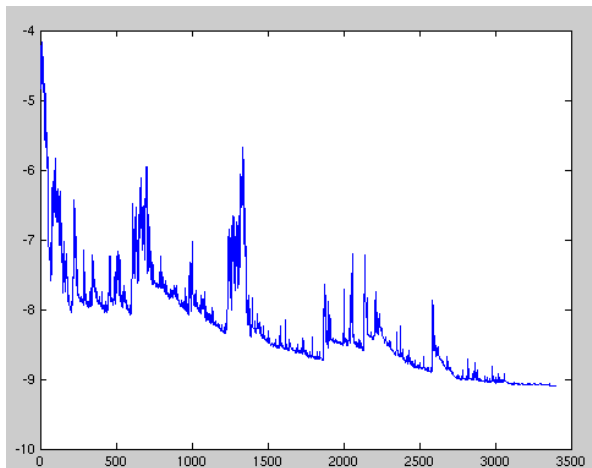
 Update Θ :

$$\Theta \leftarrow \Theta - \eta \nabla E_{\mathcal{B}}$$

$i \leftarrow i + 1$

until (convergence) OR ($i \geq MaxStep$)

If $b > 1$ $\mathcal{B}$ is called a mini-batch.



Fluctuations in the total objective function as gradient steps with respect to mini-batches are taken.

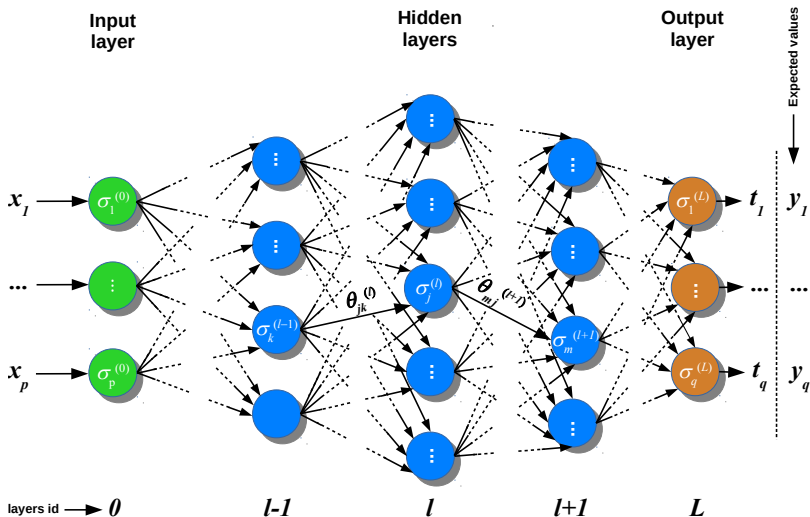
Source: <https://commons.wikimedia.org/wiki/File:Stogra.png>



3 Tuning Parameters

- General Framework
- Loss Function
- Stochastic Gradient Descent
- Backward Propagation





Let us recall that

$$\sigma_j^{(l)} = f \left(\sum_{s=1}^{n_{l-1}} \theta_{js}^{(l)} \sigma_s^{(l-1)} + b_j^{(l)} \right) = f(s_j^{(l)})$$

and if we define

$$s_j^{(l)} = \sum_{s=1}^{n_{l-1}} \theta_{js}^{(l)} \sigma_s^{(l-1)} + b_j^{(l)}$$

then

$$\sigma_j^{(l)} = f(s_j^{(l)})$$



Let also recall that

$$\nabla E_i(\Theta) = \begin{pmatrix} \frac{\partial E_i}{\partial \theta_{11}^{(1)}} \\ \frac{\partial E_i}{\partial b_1^{(1)}} \\ \vdots \\ \frac{\partial E_i}{\partial \theta_{qn_L-1}^{(L)}} \\ \frac{\partial E_i}{\partial b_q^{(L)}} \end{pmatrix}$$

so we have to compute

$$\frac{\partial E_i}{\partial \theta_{jk}^{(l)}} \text{ and } \frac{\partial E_i^{(*)}}{\partial b_j^{(l)}}$$

$\forall j, k, l.$

$(*)$: Left in exercise.



Knowing that

$$\sigma_j^{(l)} = f \left(\sum_{s=1}^{n_{l-1}} \theta_{js}^{(l)} \sigma_s^{(l-1)} + b_j^{(l)} \right) = f(s_j^{(l)})$$

we have the following chain of dependence between E_i and $\theta_{jk}^{(l)}$:

$$\theta_{jk}^{(l)} \longrightarrow s_j^{(l)} \longrightarrow \sigma_j^{(l)} \longrightarrow E_i$$

so we can use the differential chain rule:

$$\frac{\partial E_i}{\partial \theta_{jk}^{(l)}} = \frac{\partial E_i}{\partial \sigma_j^{(l)}} \cdot \frac{\partial \sigma_j^{(l)}}{\partial s_j^{(l)}} \cdot \frac{\partial s_j^{(l)}}{\partial \theta_{jk}^{(l)}}$$



$$\frac{\partial E_i}{\partial \theta_{jk}^{(l)}} = \frac{\partial E_i}{\partial \sigma_j^{(l)}} \cdot \frac{\partial \sigma_j^{(l)}}{\partial s_j^{(l)}} \cdot \frac{\partial s_j^{(l)}}{\partial \theta_{jk}^{(l)}}$$

$$\frac{\partial s_j^{(l)}}{\partial \theta_{jk}^{(l)}} = \frac{\partial}{\partial \theta_{jk}^{(l)}} \left(\sum_{s=1}^{n_{l-1}} \theta_{js}^{(l)} \sigma_s^{(l-1)} + b_j^{(l)} \right) = \frac{\partial}{\partial \theta_{jk}^{(l)}} \left(\theta_{jk}^{(l)} \sigma_k^{(l-1)} \right) = \sigma_k^{(l-1)}$$

$$\frac{\partial E_i}{\partial \theta_{jk}^{(l)}} = \frac{\partial E_i}{\partial \sigma_j^{(l)}} \cdot \frac{\partial \sigma_j^{(l)}}{\partial s_j^{(l)}} \cdot \frac{\partial s_j^{(l)}}{\partial \theta_{jk}^{(l)}}$$
$$\frac{\partial \sigma_j^{(l)}}{\partial s_j^{(l)}} = \frac{\partial f(s_j^{(l)})}{\partial s_j^{(l)}} = f'(s_j^{(l)})$$

If $f = f_s$ with $a = 1$ then

$$\begin{aligned} f'_s(x) &= \left(\frac{1}{1 + e^{-x}} \right)' = \frac{e^{-x}}{(1 + e^{-x})^2} = \frac{1}{(1 + e^{-x})} \cdot \frac{e^{-x}}{(1 + e^{-x})} \\ &= \frac{1}{(1 + e^{-x})} \cdot \frac{1 + e^{-x} - 1}{(1 + e^{-x})} = \frac{1}{(1 + e^{-x})} \cdot \left(1 - \frac{1}{(1 + e^{-x})} \right) \\ &= f_s(x) \cdot (1 - f_s(x)) \end{aligned}$$

$$\frac{\partial E_i}{\partial \theta_{jk}^{(l)}} = \frac{\partial \mathbf{E}_i}{\partial \sigma_j^{(l)}} \cdot \frac{\partial \sigma_j^{(l)}}{\partial s_j^{(l)}} \cdot \frac{\partial s_j^{(l)}}{\partial \theta_{jk}^{(l)}}$$

We must distinguish two cases :

Case 1: $l = L$ (last layer),

Case 2: $0 < l < L$ (other layers).



$$\frac{\partial E_i}{\partial \theta_{jk}^{(l)}} = \frac{\partial \mathbf{E}_i}{\partial \sigma_j^{(l)}} \cdot \frac{\partial \sigma_j^{(l)}}{\partial s_j^{(l)}} \cdot \frac{\partial s_j^{(l)}}{\partial \theta_{jk}^{(l)}}$$

Case 1: $l = L$ (last layer),

In this case $\sigma_j^{(l)} = t_j$ and then

$$\begin{aligned} \frac{\partial E_i}{\partial \sigma_j^{(l)}} &= \frac{\partial E_i}{\partial t_j} = \frac{\partial}{\partial t_j} \sum_{r=1}^q (t_r - y_r)^2 \\ &= \frac{\partial (t_j - y_j)^2}{\partial t_j} = 2(t_j - y_j) \end{aligned}$$



$$\frac{\partial E_i}{\partial \theta_{jk}^{(l)}} = \frac{\partial \mathbf{E}_i}{\partial \sigma_j^{(l)}} \cdot \frac{\partial \sigma_j^{(l)}}{\partial s_j^{(l)}} \cdot \frac{\partial s_j^{(l)}}{\partial \theta_{jk}^{(l)}}$$

Case 2: $0 < l < L$ (other layers).

In this case we can take into account the fact that

$$E_i = E_i(\sigma_1^{(l+1)}(\sigma_j^{(l)}), \dots, \sigma_{n_{l+1}}^{(l+1)}(\sigma_j^{(l)}))$$

and then using the total derivative formula

$$\frac{\partial E_i}{\partial \sigma_j^{(l)}} = \sum_{m=1}^{n_{l+1}} \frac{\partial E_i}{\partial \sigma_m^{(l+1)}} \cdot \frac{\partial \sigma_m^{(l+1)}}{\partial \sigma_j^{(l)}}$$



$$\frac{\partial E_i}{\partial \theta_{jk}^{(l)}} = \frac{\partial \mathbf{E}_i}{\partial \sigma_j^{(l)}} \cdot \frac{\partial \sigma_j^{(l)}}{\partial s_j^{(l)}} \cdot \frac{\partial s_j^{(l)}}{\partial \theta_{jk}^{(l)}}$$

Case 2: $0 < l < L$ (other layers).

In this case we can also take into account the fact that

$$E_i = E_i(\sigma_1^{(l+1)}(s_1^{(l+1)}(\sigma_j^{(l)})), \dots, \sigma_{n_{l+1}}^{(l+1)}(s_{n_{l+1}}^{(l+1)}(\sigma_j^{(l)})))$$

and then, finally

$$\frac{\partial E_i}{\partial \sigma_j^{(l)}} = \sum_{m=1}^{n_{l+1}} \frac{\partial E_i}{\partial \sigma_m^{(l+1)}} \cdot \frac{\partial \sigma_m^{(l+1)}}{\partial s_m^{(l+1)}} \cdot \frac{\partial s_m^{(l+1)}}{\partial \sigma_j^{(l)}}$$



$$\frac{\partial E_i}{\partial \sigma_j^{(l)}} = \sum_{m=1}^{n_{l+1}} \frac{\partial E_i}{\partial \sigma_m^{(l+1)}} \cdot \frac{\partial \sigma_m^{(l+1)}}{\partial s_m^{(l+1)}} \cdot \frac{\partial s_m^{(l+1)}}{\partial \sigma_j^{(l)}}$$

$$\frac{\partial s_m^{(l+1)}}{\partial \sigma_j^{(l)}} = \frac{\partial}{\partial \sigma_j^{(l)}} \left(\sum_{s=1}^{n_l} \theta_{ms}^{(l+1)} \sigma_s^{(l)} + b_m^{(l+1)} \right) = \frac{\partial}{\partial \sigma_j^{(l)}} \left(\theta_{mj}^{(l+1)} \sigma_j^{(l)} \right) = \theta_{mj}^{(l+1)}$$

$$\frac{\partial \sigma_m^{(l+1)}}{\partial s_m^{(l+1)}} = f'(s_m^{(l+1)}) \left[= \sigma_m^{(l+1)} \cdot (1 - \sigma_m^{(l+1)}) \right]_{f_s}$$

$$\frac{\partial E_i}{\partial \sigma_m^{(l+1)}} \implies \text{computed if we start from layer L toward layer 1}$$

Due to last point we talk of **backpropagation algorithm**.

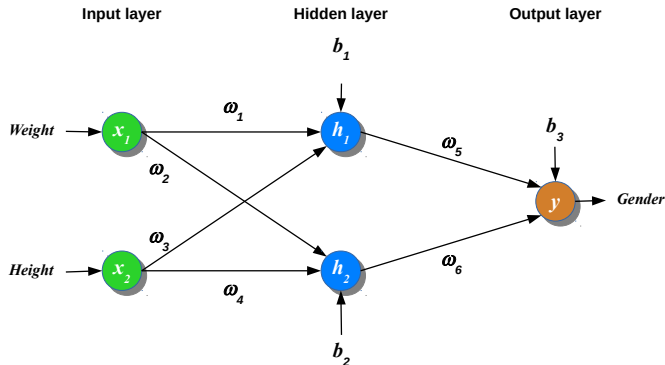


Backpropagation Algorithm :

```
for  $l = L$  to 1 do
  for  $j = 1$  to  $n_l$  do
    for  $k = 1$  to  $n_{l-1}$  do
      Compute  $\frac{\partial E_i}{\partial \theta_{jk}^{(l)}}$ 
    end for
    Store partial result  $\frac{\partial E_i}{\partial \sigma_j^{(l)}}$ 
  end for
end for
```

- 1 Introduction
- 2 Neural Networks
- 3 Tuning Parameters
- 4 Toy Example





Firstly

Name	Weight(lb)	Height(in)	Gender
Alice	133	65	F
Bob	160	72	M
Charlie	152	70	M
Diana	120	60	F
μ/σ	141.25/18.13	66.75/5.37	-

Centering and scaling numeric values and recoding objective variable (to binary).

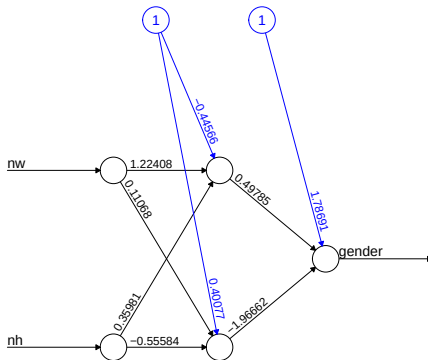
Name	SW	SH	Gender
Alice	-0.45	-0.32	1
Bob	1.03	0.97	0
Charlie	0.59	0.60	0
Diana	-1.17	-1.25	1



Then normalizing to fit into $[0, 1]$ using

$$x_n \leftarrow \frac{x_s - \min_{x_s}}{\max_{x_s} - \min_{x_s}}$$

Name	NW	NH	Gender
Alice	0.32	0.42	1
Bob	1.00	1.00	0
Charlie	0.80	0.83	0
Diana	0.00	0.00	1



Error: 0.668283 Steps: 1



Name	NW	NH	Gender	t	round(t)
Alice	0.32	0.42	1	0.83	1
Bob	1.00	1.00	0	0.06	0
Charlie	0.80	0.83	0	0.13	0
Diana	0.00	0.00	1	0.95	1